

製品概要およびセキュリティアーキテクチャ

# ODRE PQC v0.2.9

Application-Embedded Post-Quantum Security Boundary

統合体験 ・ セキュリティ境界 ・ 運用構造

JAPANESE EDITION

2026年9月

# 文書構成

|    |                                    |
|----|------------------------------------|
| 01 | 1. 製品概要                            |
| 02 | 2. 最小統合エクスペリエンス                    |
| 03 | 3. 製品アーキテクチャ                       |
| 04 | 4. Fail-Closedセキュリティ契約             |
| 05 | 5. Platform AdapterとNative Runtime |
| 06 | 6. 運用診断                            |
| 07 | 7. WorkerとLifecycleアーキテクチャ         |
| 08 | 8. Trust Boundaryと攻撃面              |
| 09 | 9. 配備アーキテクチャ                       |
| 10 | 10. 製品アイデンティティ                     |
| 11 | 11. 結論                             |

# 製品概要およびセキュリティアーキテクチャ

## 1. 製品概要

### 1.1 定義

ODRE PQCは、FastAPIアプリケーション内に導入するApplication-Embedded Post-Quantum Security Boundaryである。開発者が暗号プリミティブ、ハンドシェイク、セッション、リプレイ防御、ネイティブランタイム検証を個別に組み立てる代わりに、保護対象のリクエストがアプリケーションのハンドラーへ到達する前に、一つの境界で必要な条件を検証する。

本製品はTLS、VPN、WAFを置き換えるものではない。これらがネットワーク経路、トンネル、入口トラフィックを保護するのに対し、ODRE PQCはアプリケーションが「このリクエストを保護済みとして処理してよいか」を判断する内部境界を提供する。保護条件を確認できない場合、保護ルートは通常処理へフォールバックせず閉じる。

### 1.2 解決する問題

PQCライブラリを導入するだけでは、実運用の保護経路は完成しない。鍵交換と署名が正しくても、セッション境界、順序、重複排除、メソッドとパスの結合、レスポンス結合、ワーカー再起動時の状態、ロードされたネイティブオブジェクトの同一性が曖昧なら、アプリケーション層には攻撃可能な隙間が残る。

ODRE PQCは、これらを単一の実行契約として扱う。暗号成功だけでなく、Admission、Handshake、Session、Sequence、Replay、Request Binding、Handler実行、Response Bindingまでの連続した状態を検証し、いずれかが成立しなければハンドラーを呼び出さない。

### 1.3 中核となる価値

- [install\(app\)](#)を中心とした最小統合
- 公開ルートと保護ルートの明確な分離
- ML-KEM-768、ML-DSA-65およびHybrid Handshake
- セッション単位の厳格なSequence・Replay防御
- メソッド、パス、本文、セッションを結ぶRequest Binding
- リクエスト文脈に結び付いたResponse Binding
- Windows DLLとUbuntu SOに対するNative Runtime Trust
- 初期化、診断、検証、状態確認を分離した運用CLI
- 障害、再起動、並行初期化を含むFail-Closed Lifecycle

### 1.4 対象環境

主な対象はFastAPIを利用するAPIサーバー、エージェント、ゲートウェイ、B2Bサービス、機密業務アプリケーションである。検証対象プラットフォームはWindows Server 2022 AMD64とUbuntu 24.04 LTS AMD64であり、同一のPython Wheelと各プラットフォーム専用のNative Runtimeを組み合わせる。

### 1.5 TLS・VPN・WAF・PQCライブラリとの役割分担

| 技術       | 主な保護位置   | ODRE PQCとの関係                      |
|----------|----------|-----------------------------------|
| TLS      | 通信チャネル   | 併用する。ODRE PQCはアプリケーションの保護判断を追加する  |
| VPN      | ネットワーク境界 | 併用可能。内部ネットワークでもリクエスト単位の境界を維持する    |
| WAF      | HTTP入口   | 既知パターンやポリシーを補完し、暗号文脈とセッション状態を検証する |
| PQCライブラリ | 暗号プリミティブ | 利用するが、ODRE PQCはプロトコルと運用境界まで構成する   |

## 1.6 公開ルートと保護ルート

ヘルスチェック、公開説明、初期接続に必要なエンドポイントと、業務データを処理する保護ルートを同一のルールで扱わない。公開ルートは明示的な許可リストに置き、保護ルートはODRE PQC境界を通過した場合だけハンドラーへ接続する。設定漏れを防ぐため、既定値は「不明なルートを自動で保護済みとみなさない」である。

# 2. 最小統合エクスペリエンス

## 2.1 インストールフロー

```
from fastapi import FastAPI
from odre_pqc import install

app = FastAPI()
install(app)
```

この短い呼び出しの背後で、設定検証、Native Runtime確認、保護ルート境界、例外処理、Lifecycle Hookが接続される。暗号設定の細部を各ハンドラーに分散させず、アプリケーション起動時に一度だけ境界を構築する。

## 2.2 FastAPI統合点

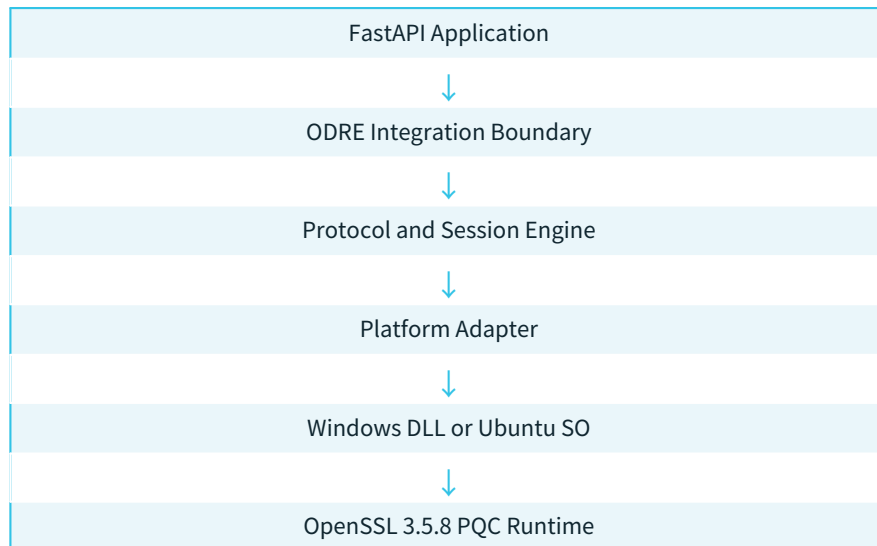
統合層はASGI/FastAPIのリクエスト処理に入り、保護対象かどうかを判定する。公開ルートなら通常のルーティングへ渡し、保護ルートならAdmissionから検証を開始する。成功時のみ元のハンドラーを一度呼び出し、戻り値を同一のリクエスト文脈へ結合して返す。

## 2.3 アプリケーション側の責任

ODRE PQCは業務認可、ユーザー権限、入力値の業務妥当性、データベース整合性を代行しない。アプリケーションは保護ルートの指定、秘密情報の安全な供給、ユーザー認証・認可、監査ログの保管、更新運用を担当する。ODRE PQCの責任は、指定された保護経路に対して暗号・プロトコル・ランタイムの条件を一貫して強制することである。

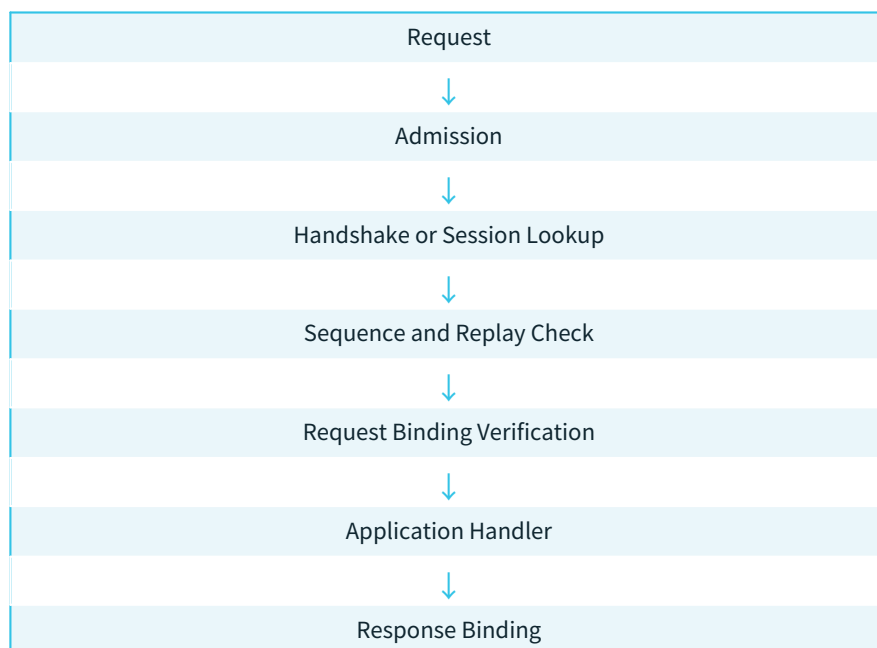
# 3. 製品アーキテクチャ

## 3.1 レイヤーモデル



アプリケーション層は業務ハンドラーを保持する。Integration Boundaryは公開・保護経路を分ける。Protocol and Session EngineはHandshake、Session、Sequence、Replay、Bindingを担当する。Platform AdapterはOS固有のロードとファイル同一性を抽象化し、Native Runtimeは実際のPQC演算を提供する。

## 3.2 保護リクエストの処理パイプライン



各段階は次段階の前提条件である。前段階が失敗した場合は、その時点で処理を停止し、後続の状態を生成しない。特にHandlerは全検証の後にのみ呼び出され、並行リクエストでも単一実行性を維持する。

## 3.3 Admission

Admissionはリクエストが公開経路か保護経路か、必要な構造とサイズ制限を満たすか、既知のプロトコルバージョンかを確認する。MalformedまたはOversized Inputは暗号処理や業務処理へ渡す前に拒否し、資源消費と解析の曖昧性を抑える。

## 3.4 HandshakeとSession

新規接続はHandshakeでアルゴリズム、エフェメラル材料、署名検証、セッション識別子を確立する。既存セッションでは識別子だけでなく、有効期限、状態、所有するワーカー文脈、直前のSequenceを確認する。Handshakeの途中状態は完了済みSessionとして利用できない。

### 3.5 暗号方式の役割

ML-KEM-768は共有秘密の確立に、ML-DSA-65は署名と真正性確認に用いる。Hybrid構成は古典方式とPQC方式の両条件を契約どおり評価する。必要なPQC ProviderやNative Runtimeを確認できなければ、保護経路を弱い方式へ自動Downgradeしない。

### 3.6 Session Lifecycle

Sessionは生成中、Active、Closing、Expired、Invalidなどの状態を持つ。状態遷移は原子的に処理し、CrashやRestartの後に古いLeaseや中間状態を新しいSessionとして再利用しない。Shutdown時は新規Admissionを止め、進行中の処理を境界内で完了または安全に中止する。

### 3.7 厳格なSequenceとReplay防御

各保護メッセージはSession内の期待Sequenceと比較される。過去値、重複値、許可されない飛び越しは拒否する。比較と更新を同じ排他境界で実行し、二つの並行リクエストが同一Sequenceを同時に成功させない。

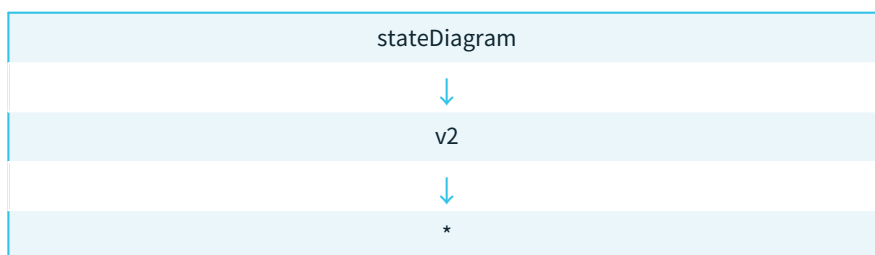
### 3.8 Request Semantic Binding

暗号化されたPayloadだけでなく、HTTPメソッド、正規化されたPath、Session ID、Sequence、必要なメタデータを同一の認証対象へ結合する。これにより、あるルート向けの有効なメッセージを別ルートや別メソッドへ移す攻撃を防ぐ。

### 3.9 Response Binding

レスポンスは対応するRequest ID、Session、Sequenceおよび結果文脈へ結合される。クライアントは、暗号的に有効であっても別のリクエストから取得したレスポンスを現在の結果として受け入れない。

## 4. Fail-Closedセキュリティ契約



Fail-Closedとは、エラーを返すことだけではない。検証失敗時にHandlerを呼ばない、部分的なSessionを残さない、未検証のNative Runtimeへ切り替えない、状態表示を認可として使わない、再起動後に古い状態を復活させない、という一連の契約である。

エラーは運用者が原因を識別できる粒度で記録するが、外部応答には秘密、ローカルパス、鍵材料、詳細な内部状態を露出しない。可観測性と情報最小化を同時に維持する。

## 5. Platform AdapterとNative Runtime

### 5.1 クロスプラットフォーム境界

Python層は共通のセキュリティ契約を実装し、Platform AdapterがOS固有のロード、ファイル属性、プロセスモデルを扱う。同一Wheelを利用しながら、Windows DLLとUbuntu SOはそれぞれのManifest、Archive、SHA-256、ロード済みオブジェクト情報で識別される。

### 5.2 Windows Native Trust

Windowsでは通常ファイル、Junction、Symlink、Reparse Pointを区別する。ロード前のパス確認だけでなく、実際にロードされたModuleの情報と期待値を比較する。検証対象が途中で置換された場合、保護経路の初期化を失敗させる。

### 5.3 Ubuntu Native Trust

Ubuntuではファイルディスクリプタ、device、inode、所有権、権限、実体パスを用いて、確認したオブジェクトと利用するオブジェクトの関係を追跡する。Symlink交換やTOCTOU競合が生じた場合、同一性を証明できないロードを受け入れない。

### 5.4 Runtime ArchiveとManifest

配布Archiveは内容一覧とハッシュを持つManifestに結び付く。展開先は境界外へのPath Traversalを許可せず、既存ファイルとの衝突、権限、期待外の追加ファイルを検査する。Manifest、Archive、展開結果、Loaded Objectの四段階を連結して確認する。

## 6. 運用診断

| コマンド                | 目的                           | セキュリティ上の意味           |
|---------------------|------------------------------|----------------------|
| <code>init</code>   | 必要な設定とRuntimeを準備             | 初期化を明示し、暗黙の生成を避ける    |
| <code>doctor</code> | 環境・依存関係・権限を診断                | 問題の位置を運用者へ提示する       |
| <code>verify</code> | Artifact、Manifest、Runtimeを検証 | 保護開始前の信頼条件を確認する      |
| <code>status</code> | 現在の動作状態を表示                   | 可観測性を提供するが認可判断には使わない |

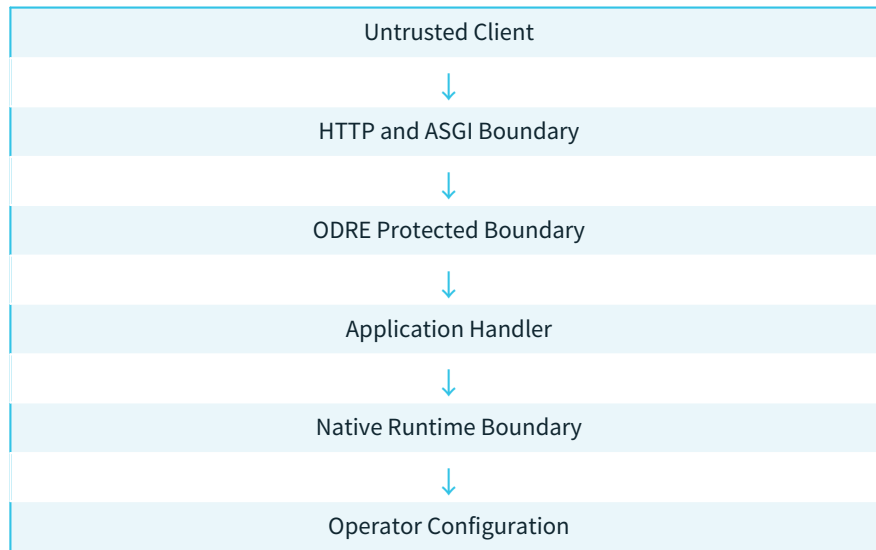
診断結果は成功・失敗だけでなく、どの境界で条件が不足したかを示す。ただし、表示上のHealthyは保護リクエストの検証を省略する根拠にはならない。各リクエストは自身のSessionとSequence条件を通過する必要がある。

## 7. WorkerとLifecycleアーキテクチャ

複数のUvicorn Workerが同時に起動する環境では、Runtime展開、初期化、Lease、Session所有権が競合し得る。ODRE PQCはプロセス間ロックと原子的な状態更新を用い、一つのWorkerだけが共有初期化を完了し、他のWorkerは完成した同一結果を検証して利用する。

Crash時には「起動中」と「使用可能」を区別する。途中で終了したOwnerのLeaseを期限とIdentityで判定し、単なるPID再利用を所有権の証明として扱わない。Restart後はStale Sessionを排除し、Native RuntimeとManifestを再確認してから保護経路を開く。

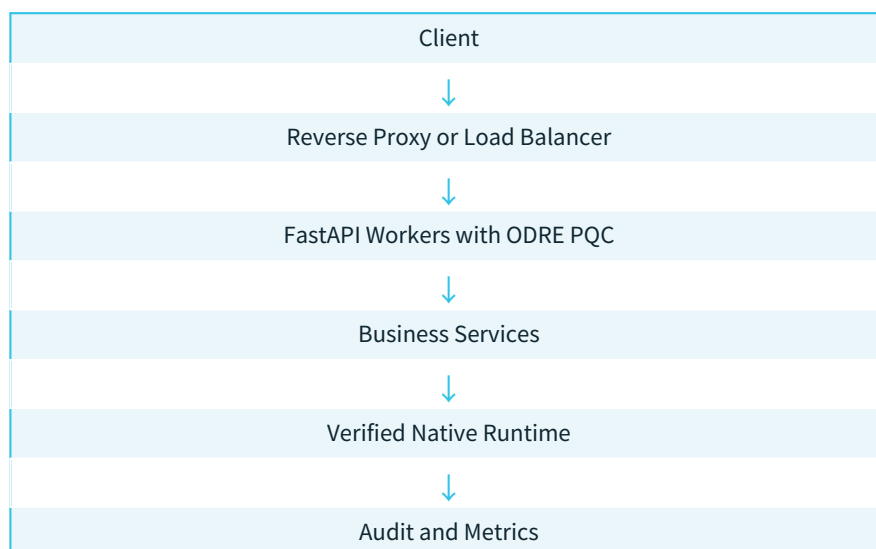
## 8. Trust Boundaryと攻撃面



外部クライアント、HTTPパーサー、設定供給元、ファイルシステム、ネイティブLoader、Worker間共有状態はそれぞれ異なる信頼レベルを持つ。攻撃面にはMalformed Input、Replay、Cross-Session、Path交換、Provider Injection、Config Injection、Concurrent Init、Crash後のStale Stateが含まれる。

ODRE PQC境界を通過しても、業務ハンドラー内部のSQL Injection、権限誤設定、データ漏えいが自動的に解決されるわけではない。境界の内側でもSecure Coding、最小権限、監査、秘密管理が必要である。

## 9. 配備アーキテクチャ



Reverse ProxyはTLS終端、ルーティング、基本的なトラフィック制御を担当できる。ODRE PQCは各FastAPI Worker内で保護判断を行うため、Proxyを通過したこと自体を保護済みの証明にしない。Trusted Proxy設定を利用する場合は、信頼するHopと転送Headerを明示する。



---

配備単位はWheel、プラットフォーム用Runtime Archive、Manifest、設定、秘密供給機構から構成される。Runtimeや設定をコンテナImageへ含める場合も、起動時のIdentity検証を維持する。ログとメトリクスはSession IDを必要最小限に扱い、鍵や平文Payloadを記録しない。

## 10. 製品アイデンティティ

1 Unitは、独立した一つのアプリケーションインストールを意味する。同じインストールの複数Workerは、ライセンス契約で別途定めない限り、暗号・Lifecycle上は一つの配備単位として扱う。異なるサーバー、コンテナ群、顧客環境への複製は、運用・ライセンス設計に従って個別に識別する。

製品バージョン、Wheel SHA-256、Runtime Manifest、Native Runtime SHA-256、対象OS、Python、OpenSSLバージョンを一組で記録する。表示名だけでは実行中の製品を一意に特定できないため、診断と監査ではArtifact Identityを用いる。

## 11. 結論

ODRE PQC v0.2.9は、PQCアルゴリズムを呼び出すための薄いWrapperではない。FastAPIアプリケーションの保護ルート前面に、Admission、Handshake、Session、Sequence、Replay、Request/Response Binding、Native Runtime Trust、Lifecycleを一つのFail-Closed契約として配置する製品である。

導入者は`install(app)`を中心に境界を接続し、公開・保護ルート、秘密供給、業務認可、運用監視を構成する。製品はWindows Server 2022およびUbuntu 24.04 LTS向けのPlatform Adapterを通じて同じ上位契約を維持する。技術白書はこのアーキテクチャの開発・修正・検証系譜を別文書として提示する。